TP3 - Projet Cloud Computing

Analyse de sentiment des commentaires YouTube

Questions de réflexion

Élément de Module : Industrie 4.0

Réalisé par : Ilyas Fardaoui

Encadré par : Prof. Maria Zemzami

Université Mohamed V - ENSAM Rabat
Filière INDIA | Année universitaire 2026/27

PHASE 1

Phase 1 : Configuration de l'environnement MLOps

```
EXPLORER  —  Youtube-Sentiment-MLops-Project

  data/
     raw/reddit.csv
     processed/  (train.csv, test.csv, class_distribution.png)
  src/
     data/    download_dataset.py, prepare_data.py
     models/  train.py
     api/     main.py
  models/      *.joblib (pipeline, vectoriseur), metrics.json
  chrome-extension/  manifest.json, popup.html/css/js, content.js
  tests/       test_api.py, test_model.py
  docs/        API.md, rapport_technique.md, screenshots/
  logs/        .gitkeep
  Dockerfile
  app_api.py
  requirements.txt   requirements-prod.txt
  README.md
```

Figure 1.1 : Structure du projet.

PHASE 1

Phase 1 : Questions de réflexion

1 - Pourquoi séparer les données raw et processed ?

data/raw conserve les données brutes intactes : si le nettoyage change, on régénère data/processed sans retélécharger ni altérer la source.

Cela garantit un flux unidirectionnel (raw -> nettoyage -> processed) et permet à l'entraînement de charger directement les splits nettoyés sans rejouer le nettoyage.

2 - Quels fichiers ne doivent jamais être commités sur Git ?

L'environnement virtuel (.venv/), les secrets (.env, tokens), les datasets volumineux (data/raw et data/processed/*.csv), les caches et artefacts (__pycache__/, *.pyc, logs/), et les gros modèles. Mon .gitignore couvre ces cas.

3 - Quelle est l'importance d'un environnement virtuel ?

Il isole les dépendances du projet pour éviter les conflits de versions entre projets, protège l'installation Python globale, et rend l'environnement reproductible via requirements.txt.

PHASE 2

Phase 2 : Pipeline de données et préparation

Classe	Total nettoyé	Train (80%)	Test (20%)	Long. moy. (mots)	Long. méd. (mots)
Positif (1)	15 723 (43,0%)	12 578	3 145	42,9	21
Neutre (0)	12 606 (34,5%)	10 085	2 521	9,9	7
Négatif (-1)	8 233 (22,5%)	6 586	1 647	34,9	20
Total	**36 562**	**29 249**	**7 313**	**-**	**-**

Tableau 2.1 : Répartition et longueur des commentaires (après nettoyage).

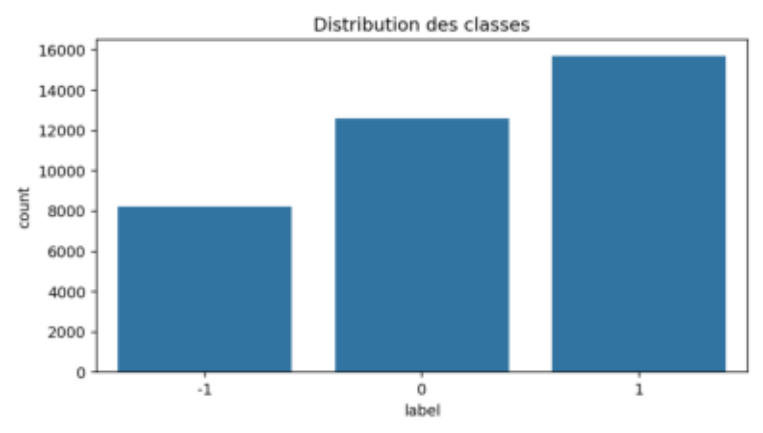


Figure 2.1 : EDA — distribution des classes.

PHASE 2

Phase 2 : Questions de réflexion

1 - Pourquoi nettoyer les URLs et mentions ?

Les URLs et mentions sont quasi uniques et créent des milliers de tokens rares sans valeur sémantique, qui gonflent la matrice TF-IDF. Les remplacer par des tokens (URL, USER, HASHTAG) réduit le bruit et la dimensionnalité et limite le surapprentissage.

2 - Comment gérer les emojis dans le texte ?

Dans mon nettoyage, la normalisation retire les emojis avec les autres symboles pour privilégier le signal textuel. Une amélioration serait de les convertir en tokens sémantiques (emoji_positif, emoji_négatif) avant la normalisation, afin de conserver ce signal émotionnel utile sur les commentaires courts.

3 - Quel ratio train/test est approprié ?

Un split stratifié 80/20 (random_state=42) : 29 249 commentaires pour l'apprentissage et 7 313 pour une évaluation fiable (dont plus de 1 600 pour la classe minoritaire). La stratification préserve la distribution des classes.

PHASE 3

Phase 3 : Développement et entraînement du modèle

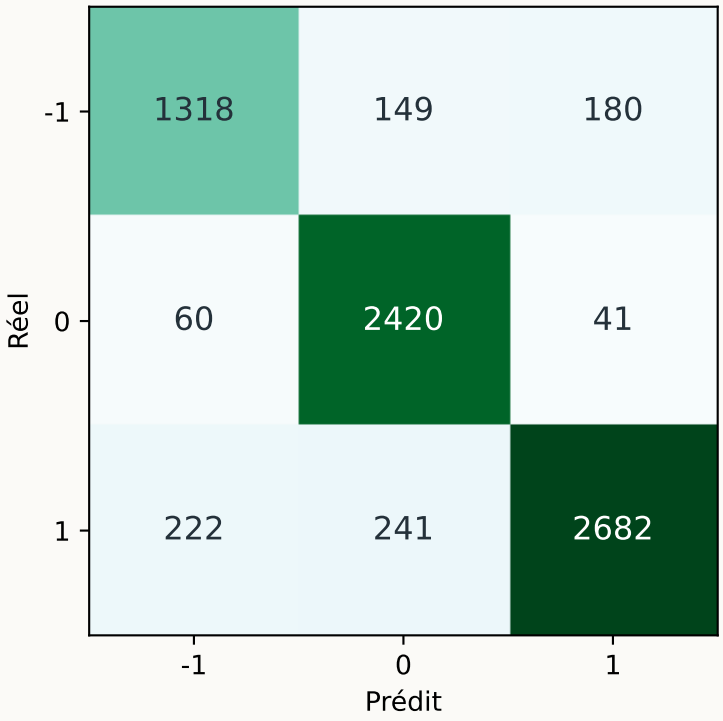


Figure 3.1 : Matrice de confusion (jeu de test, 7 313 commentaires).

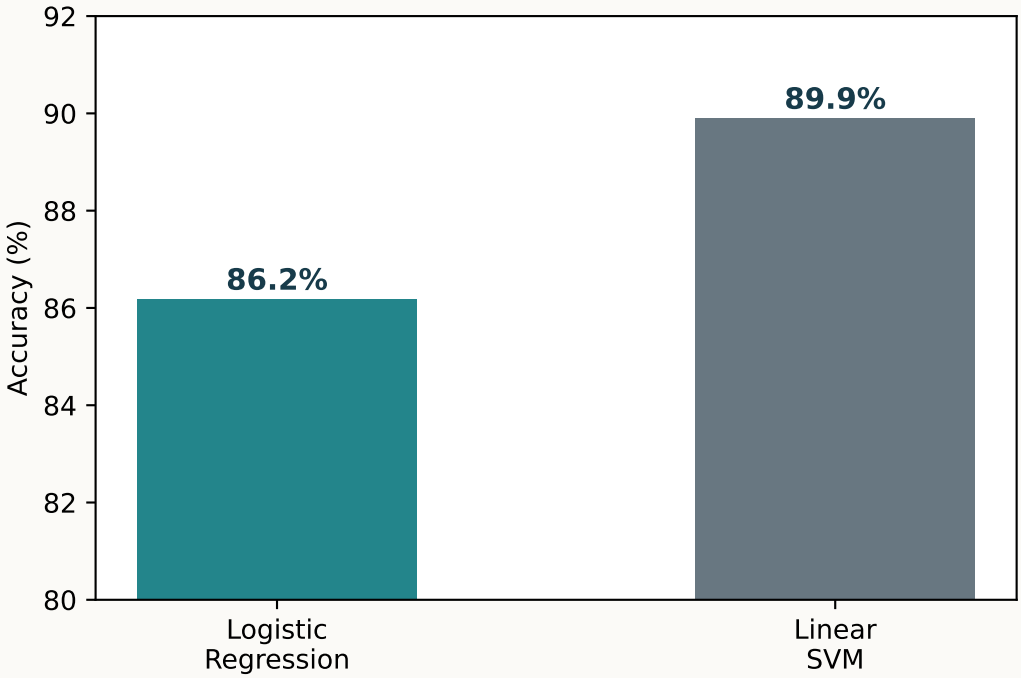


Figure 3.2 : Comparaison des modèles — LR retenue pour predict_proba.

PHASE 3

Phase 3 : Questions de réflexion

1 - Pourquoi utiliser TF-IDF plutôt que Count Vectorizer ?

CountVectorizer donne un poids élevé aux mots fréquents mais peu informatifs (the, video). TF-IDF pondère la fréquence locale par la rareté globale (IDF) et met en valeur les termes discriminants porteurs d'opinion (amazing, horrible, waste).

2 - Comment interpréter la matrice de confusion ?

La diagonale (1318 + 2420 + 2682 = 6420 sur 7313, soit 87,79 %) représente les prédictions exactes. La classe Neutre a le meilleur rappel (96,0 %), devant Positif (85,3 %) et Négatif (80,0 %). Les confusions opposent négatif et positif aux neutres, sur des commentaires ambigus ou ironiques.

3 - Quels hyperparamètres ont le plus d'impact ?

GridSearchCV a retenu C=2.0, min_df=2, ngram_range=(1, 1). class_weight='balanced' corrige le déséquilibre de la classe négative (F1 porté à 81,18 %). C ajuste la régularisation L2.

PHASE 4

Phase 4 : Développement de l'API REST

1 - Pourquoi utiliser FastAPI plutôt que Flask ?

FastAPI est asynchrone (ASGI/Uvicorn) et plus rapide que Flask (WSGI) pour servir des prédictions en temps réel. Il valide les entrées avec Pydantic (erreurs 422 explicites) et génère automatiquement Swagger (/docs).

2 - À quoi sert la configuration CORS ?

La Same-Origin Policy du navigateur bloque par défaut les appels fetch() d'une extension ou de youtube.com vers un serveur externe. Le middleware CORS ajoute les en-têtes Access-Control-Allow-Origin/Methods nécessaires pour autoriser l'extension à appeler l'API.

3 - Comment gérer efficacement le chargement du modèle ?

Le pipeline .joblib est chargé une seule fois en mémoire au démarrage du module de l'API (variable globale), et non rechargé à chaque requête. Chaque appel réutilise l'objet en RAM, garantissant une latence faible (environ 4 ms pour 50 commentaires).

PHASE 5

Phase 5 : Développement de l'extension Chrome

1 - Comment gérer les commentaires avec des réponses (threads) ?

Mon content script extrait les commentaires principaux (ytd-comment-thread-renderer #content-text). Les réponses sont dans ytd-comment-replies-renderer ; une amélioration serait une case 'Inclure les réponses' pour analyser aussi les discussions.

2 - Quelle stratégie pour extraire les commentaires dynamiques ?

YouTube charge les commentaires en lazy-loading au défilement. L'extension lit ceux présents dans le DOM à l'analyse (run_at document_idle) ; l'utilisateur fait défiler pour en charger davantage puis relance. Un défilement automatique est une amélioration prévue.

3 - Comment optimiser les performances avec de nombreux commentaires ?

Les commentaires visibles (plafonnés à 100) sont envoyés en une seule requête /predict_batch et vectorisés en une passe. Les filtres par sentiment opèrent côté client, en mémoire dans popup.js, sans réinterroger le serveur.

PHASE 6

Phase 6 : Déploiement cloud avec Docker

1 - Pourquoi séparer les requirements dev et prod ?

pandas, matplotlib, seaborn et pytest pèsent plusieurs centaines de Mo et ne servent pas à l'inférence HTTP. requirements-prod.txt ne garde que fastapi, uvicorn, scikit-learn, pandas, numpy et joblib : image plus légère, moins de surface d'attaque et moins de RAM.

2 - Comment réduire la taille de l'image Docker ?

Image de base python:3.10-slim, pip install --no-cache-dir, copie des seuls fichiers nécessaires (app_api.py, src, models), et ordre des couches optimisé (COPY requirements avant le code) pour maximiser le cache.

3 - Quelle stratégie pour les mises à jour du modèle ?

Réentraîner via le pipeline reproductible, valider les métriques avant promotion (pas de régression), versionner les .joblib, puis reconstruire et redéployer l'image — idéalement via une CI/CD.

4 - Cloud éphémère (Codespaces) vs PaaS permanent ?

Un Codespace s'arrête après ~30 min d'inactivité et doit être relancé avant chaque démonstration ; un PaaS reste disponible en continu avec une URL stable, au prix d'un hébergement payant. Pour une vraie production, le PaaS (ou un conteneur managé) est préférable pour la disponibilité.

PHASE 7

Phase 7 : Tests et validation

```
$ pytest -q

tests/test_api.py .......                                [ 54%]
tests/test_model.py ......                               [100%]

================= 13 passed in 19.40s =================

API   : health, batch vide, hors-limite, payload invalide,
        commentaire vide, prédiction cohérente, batch de 50
Modèle: accuracy >= 80%, F1 par classe > 75%, latence < 100ms,
        cas limites (court/long, emojis, langues mélangées)
```

Figure 7.1 : Exécution de la suite de tests (13 tests passés).